

Anomaly detection means identifying **data points, records, patterns, or behavior that are unusual compared to normal data.**

In your case, your data feed contains event, attendee, product, country, and activity information.

An anomaly means something like:

- sudden drop in records
- missing important values
- duplicate IDs
- abnormal country distribution
- unusual attendee counts
- unexpected event patterns
- schema changes
- corrupted values
- delayed feeds
- statistical deviations

This is extremely important in enterprise data engineering because anomaly detection helps detect:

- ETL failures
- source system issues
- API problems
- business process issues
- data corruption
- fraud/risk patterns
- integration failures

1. What is Anomaly?

Simple definition:

An anomaly is data that behaves differently from expected normal behavior.

Example from your dataset:

Scenario	Normal	Anomaly
Daily Row Count	100k rows/day	Suddenly 5k rows
Country Values	US, IN, UK	NULL or UNKNOWN spikes
Event Dates	Proper range	<code>event_end_date < event_start_date</code>
Product IDs	Mostly unique mapping	Same <code>product_id</code> linked to many unrelated products
Completeness	98% filled	Suddenly 40% nulls
Attendee Status	Accepted/Declined	Random unexpected status
Column Length	<code>event_name</code> length < 200	Suddenly 10,000 chars
Distinct Count	Stable	Sudden explosion/drop

2. Types of Anomalies

There are mainly 3 major anomaly categories.

A. Point Anomaly

A single record is abnormal.

Example:

event_id	attendee_count
E1	120
E2	130
E3	15000 ← anomaly

Very common in:

- transaction systems
- row-level validation
- fraud detection

B. Contextual Anomaly

Value is abnormal only in a certain context.

Example:

- 100 attendees may be normal for US
- but abnormal for a small local country event

OR

- High row count normal on Monday
- abnormal on Sunday

Context matters:

- date
- geography
- product
- business unit

- seasonality
-

C. Collective Anomaly

A group of records together becomes abnormal.

Example:

- One missing attendee is okay
- 40% attendee records missing together = anomaly

OR

- Many events suddenly changing to CANCELLED

This is common in:

- pipeline failures
 - upstream API failures
 - schema issues
-

3. Types of Data Quality Anomalies in Your Dataset

Your dataset is enterprise event + attendee data.

You can detect anomalies in:

A. Volume Anomalies

Check:

- RowCount
- daily ingestion count
- country-wise counts
- event-wise counts

Example:

- Daily rows suddenly drop by 80%

Metrics:

- RowCount
 - Distinct ValuesCount
 - ColumnCount
-

B. Completeness Anomalies

Check NULL percentages.

Critical columns:

- event_id
- product_id
- attendee_status
- event_start_date

Example:

- attendee_country suddenly becomes 70% NULL

Metrics:

- Completeness
-

C. Uniqueness Anomalies

Check duplicate IDs.

Example:

- event_id duplicated unexpectedly

Metrics:

- Uniqueness
 - Distinct ValuesCount
 - Unique ValueRatio
-

D. Schema Anomalies

Detect:

- missing columns
- new columns
- datatype changes

Example:

- event_start_date becomes string instead of timestamp

Metrics:

- ColumnCount

- ColumnLength
-

E. Statistical Anomalies

Detect distribution changes.

Example:

- Mean attendee count shifts heavily
- Entropy changes drastically

Metrics:

- Mean
 - StandardDeviation
 - Entropy
 - Sum
-

F. Business Rule Anomalies

Most important in enterprise systems.

Examples:

Date anomalies

`event_end_date < event_start_date`

Country mismatch

`event_country != attendee_country`

Invalid status

attendee_status NOT IN ('Accepted','Declined','Pending')

Invalid mappings

product_id linked to multiple unrelated products

Metrics:

- CustomSql
 - ColumnCorrelation
-

4. Best Anomaly Detection Approaches For Your Data

Your data is mostly:

- structured
- tabular
- enterprise feed data
- relational

So you should use:

LEVEL 1 — Rule Based Detection (MOST IMPORTANT)

This is the best starting point.

Enterprise companies heavily rely on this.

Examples:

NULL check

```
SELECT COUNT(*)  
FROM table  
WHERE event_id IS NULL
```

Duplicate check

```
SELECT event_id, COUNT(*)  
FROM table  
GROUP BY event_id  
HAVING COUNT(*) > 1
```

Date validation

```
WHERE event_end_date < event_start_date
```

Advantages:

- easy
- explainable
- accurate
- production-friendly

Best for:

- data engineering
 - ETL monitoring
 - pipelines
-

LEVEL 2 — Statistical Detection

Detect deviations from historical behavior.

Example:

If normal row count:

Day	Count
Mon	100k
Tue	102k
Wed	98k

Today:

- 20k rows

This becomes anomaly.

Techniques:

- Z-score
- moving average
- standard deviation
- IQR

Example:

```
genui  {"math_block_widget_always_prefetch_v2":{"content":"z = \frac{x-\mu}{\sigma}"}}
```

If:

- $|z| > 3$
→ anomaly

Good for:

- RowCount
 - Mean
 - Sum
 - Completeness %
-

LEVEL 3 — ML-Based Anomaly Detection

Use when:

- large-scale systems
- patterns are complex
- manual rules difficult

Algorithms:

Algorithm	Use Case
Isolation Forest	General tabular anomalies
One-Class SVM	Boundary detection
Autoencoder	Deep learning anomalies
DBSCAN	Density anomalies
LOF	Local outlier detection

For your use case:

- Isolation Forest is best ML starting point
-

5. Which Metrics You Should Use

From your available metrics:

Metric	Use
RowCount	volume anomaly
Completeness	null anomaly
Distinct ValuesCount	uniqueness anomaly
Uniqueness	duplicate anomaly
Mean	statistical anomaly
StandardDeviation	distribution anomaly
Entropy	pattern change
ColumnLength	corrupted strings
ColumnCorrelation	relationship anomaly
Unique ValueRatio	duplicate spikes
ColumnCount	schema anomaly
Sum	aggregate anomaly
CustomSql	business rules

6. Best Enterprise Architecture For Your Use Case

Recommended flow:

```

Raw Feed
  ↓
Data Validation Layer
  ↓
Anomaly Detection Layer
  ↓
Alerting System
  ↓
Dashboard
  ↓
Quarantine / Reject Records

```

7. Recommended Anomaly Checks For Your Dataset

Critical Checks

Feed-Level

- RowCount
 - ColumnCount
 - Duplicate rows
 - Missing partitions
-

Column-Level

event_id

- uniqueness
- completeness

event_start_date

- valid timestamp
- no future anomalies

attendee_status

- valid enum values

event_country

- accepted country codes

product_id

- mapping consistency
-

Business-Level

Date Logic

event_end_date >= event_start_date

Status Logic

Cancelled events should not have attendees marked attended

Country Logic

event_country and attendee_country relationship validation

8. Enterprise Tools Used For This

Popular tools:

Tool	Purpose
Great Expectations	Data quality
Deequ	AWS/Spark anomaly detection
Soda	Data observability
Monte Carlo	Enterprise monitoring
Datadog	Monitoring
Airflow checks	ETL anomaly detection

For your environment:

- Great Expectations

- Deequ
- custom SQL validations

are excellent.

9. Recommended Implementation Strategy

Phase 1

Start with:

- RowCount
- Completeness
- Uniqueness
- CustomSql

This gives 80% value.

Phase 2

Add:

- statistical thresholds
 - moving averages
 - z-score anomalies
-

Phase 3

Add ML:

- Isolation Forest
 - trend analysis
 - seasonality
-

10. Best Practical Example For Your Data

Example anomaly pipeline:

1. Daily feed arrives
 2. Check row count
 3. Check NULL %
 4. Check duplicates
 5. Check schema
 6. Compare against historical baseline
 7. Generate anomaly score
 8. Send alerts
 9. Store anomaly logs
-

11. Important Enterprise Insight

In real enterprise systems:

- 70% anomaly detection = rule-based
- 20% statistical
- 10% ML

Most production systems rely heavily on:

- SQL rules
- thresholds

- historical comparisons

because they are:

- explainable
- stable
- auditable
- easier to maintain

ML-only anomaly detection is rarely enough in enterprise data feeds.

12. What I Recommend Specifically For Your Dataset

Start with these immediately:

Must Have

- RowCount anomaly
 - Completeness anomaly
 - Duplicate event_id
 - Invalid date logic
 - Enum validation
 - Distinct count monitoring
-

Then Add

- entropy drift

- country distribution drift
- attendee trend anomalies
- product mapping anomalies

13. Example Enterprise Anomaly Categories For Your Dataset

Category	Example
Volume	Row count drop
Freshness	No new data
Completeness	NULL spikes
Uniqueness	Duplicate IDs
Validity	Invalid dates
Consistency	Product mismatch
Distribution	Country drift
Schema	Column changes
Relationship	Broken correlations
Statistical	Mean deviation

Final Recommendation

For your use case:

BEST STARTING APPROACH

1. Rule-based anomaly detection
2. Statistical threshold detection

3. Historical baseline comparison

4. Custom business SQL rules

Then later:

- ML anomaly detection
-

Best Metrics To Prioritize

Priority order:

1. Completeness
2. RowCount
3. Uniqueness
4. Distinct ValuesCount
5. CustomSql
6. Entropy
7. StandardDeviation
8. ColumnCorrelation

These will provide the highest business value for enterprise feed monitoring.